Development Summary - Classroom Survey Web App

Repository: https://github.com/george3281/classroom-survey
Deployed app: https://classroom-survey.onrender.com/
AI coding agent: Claude Code (Anthropic), run from the Claude desktop app

1. Using the specifications to guide the agent

I put the five provided Markdown files in the project folder and gave the agent the full assignment text, telling it to complete the assignment from those files. Before writing code, the agent read all five specs end to end. The specs drove the work like this:

- **DATABASE.md** became the migration almost verbatim: the five tables, their foreign keys and `on delete cascade` rules, the indexes, the `updated_at` trigger, the partial unique index that allows only one active survey, and the RLS policies for `anon` and `authenticated`.
- **API.md** set the shape of the data layer. Its recommended RPCs (`activate_survey`, `close_survey`, `submit_survey_response`) and its error codes (`SURVEY_NOT_ACTIVE`, `DUPLICATE_RESPONSE`, `INVALID_CHOICE`, and so on) were implemented as named, and the frontend maps each code to a friendly message.
- **UI.md** gave the routes, wireframes, empty states, confirmation dialogs, student submission states, and accessibility rules. Its exact wording ("Thank you! Your response has been recorded.", "No surveys match your search.") is used in the app.
- **TODO.md** was the checklist. After testing it was updated so it shows what is actually done. The provided copy had some deployment items already ticked, and those were unticked until they were really finished.

Where the specs left a decision open (for example, whether a closed survey can be edited, or how to tell "closed" from "not found" when RLS hides closed surveys), the agent picked an option and recorded it in the "Design decisions" table in README.md.

2. What the AI agent generated

Nearly all code was generated by the agent:

- the Vite + React + TypeScript + Tailwind project and its configuration
- `supabase/migrations/20261105000000_init.sql`, which holds the schema, RLS, and five RPC functions
- the seven pages (landing, login, dashboard, editor, present, results, student survey), the auth provider, and protected routes
- `src/lib/api.ts` (all Supabase calls) and `src/lib/results.ts` (aggregation and CSV)
- the automated tests, the README, and drafts of this summary and the AI history

My part was to set the goals, review the plan and the code, configure the external services (Supabase project, Auth settings, GitHub, Render), and test the deployed app. The agent took the database

screenshots from a local Supabase stack running the same migration.

3. Database configuration

The database is PostgreSQL hosted by Supabase. Its structure is two chains: `surveys -> questions -> choices` and `surveys -> responses -> answers`, where each answer also points to a `question` and a `choice`. Every table has a UUID primary key. Foreign keys enforce the relationships, and deleting a survey cascades to all of its data.

Security is handled in the database itself:

- **RLS is on for every table.** Anonymous students can `select` only the active survey and its questions and choices. No anonymous policy exists for `responses` or `answers`, so students can never read results.
- **Student writes go through one function.** That function is `submit_survey_response`, which runs as `security definer` with a fixed `search_path`. In a single transaction it checks that the survey is active, that each question belongs to the survey, that each choice belongs to its question, that every question is answered, and that the browser token has not already been used. Then it inserts the response and its answers.
- **Instructors are authenticated users.** Public sign-up is turned off in Supabase Auth, and the instructor account was created by hand in the dashboard.

To set up the hosted database, I ran the migration file in the Supabase SQL Editor.

4. How the frontend talks to the database

There is no custom backend. The React app uses `@supabase/supabase-js` with the project URL and the public anon key, both read from Vite environment variables. Those variables are never committed: they live in `.env.local` locally and in Render's environment settings in production.

- Reads use PostgREST queries with nested selects, for example `surveys.select('*, questions(*, choices(*))')`.
- Writes use RPC calls, so multi-row changes are atomic.
- Login uses `supabase.auth.signInWithPassword`. The JWT it returns turns requests into the `authenticated` role, and RLS then grants instructor access.

5. Problems encountered and how they were resolved

1. **The local Postgres server needed a password, so the migration couldn't be tested there.** The agent switched to PGlite, a Postgres engine that runs inside Node. `tests/db.test.mjs` builds the same `anon` and `authenticated` roles that Supabase uses, runs the real migration, and tries attacks as a student: inserting surveys, reading responses, submitting twice, submitting to a closed survey, and submitting a choice from the wrong question. Later the agent started Docker and ran the full Supabase stack locally (`npx supabase start`) to test the real Auth and REST API in a browser.

2. **A security risk in the submission function.** The first version used a temporary table inside a `security definer` function. Postgres searches the temporary schema first unless told otherwise, so a malicious caller could create a temp table named `questions` and change the function's behavior. The fix was to drop the temp table, read the JSON payload directly with `jsonb_to_recordset`, and set `search_path = public, pg_temp`.
3. **Zero-vote choices had no label in the chart.** During browser testing, a choice with 0 votes showed no "0 (0%)" label, because Recharts doesn't draw labels for bars of zero width. UI.md says counts must always be shown. The fix was `minPointSize={3}` on the bar.
4. **The bundle was large on student phones.** The first build produced one 778 KB file, mostly the Recharts library, which only instructors need. The Results page is now lazy-loaded, so students no longer download the chart code.

6. Deployment

The code is on GitHub. I chose Render instead of Cloudflare Pages (the assignment allows any suitable host). Render deploys it as a free static site from `render.yaml`: it runs `npm ci && npm run build` on Node 22 and serves the `dist` folder. `VITE_SUPABASE_URL` and `VITE_SUPABASE_ANON_KEY` are entered in the Render dashboard, not stored in the repo. A rewrite rule sends every route to `index.html`, so deep links like `/s/<slug>` work. Before deploying, the agent ran the same build in a Linux Node 22 Docker container to make sure it would succeed on Render. QR codes are built from `window.location.origin`, so in production they point to the Render URL automatically. The database stays on Supabase.

7. What I learned about full-stack development

- **The database can enforce security.** Tests showed that even with the public key in hand, a student cannot do anything that RLS and the RPCs don't allow.
- **Atomic operations matter.** Saving a survey, activating one, and submitting a response each touch several rows, so each needs to succeed or fail as a whole. Database functions provide that.
- **"It compiles" is not "it works."** The zero-count label bug only showed up when looking at real data in a browser, and the temp-table problem only showed up in review.
- **The agent works best with specific direction.** Clear specs plus a demand for end-to-end verification produced better results than asking for code alone.